SGLang self-benchmark journey

One campaign cell with two benchmark points, followed from AISim's plan to a published profile row. The original design runs in lane A and the in-scheduler variant in lane B, so each stage shows what the two share and where they part.

SHARED
What is measured, the point geometry, and every validity rule.

DIFFERENT
Where the controller runs (stages 2 and 3), which interval is timed (7 and 9), and which artifact schema is written (10 and 12).

STANDING
Lane B is a proposal from reading source. It has not been prototyped, and nothing here was run on a GPU.

A · Original design
A standalone runner outside the scheduler, as written in the design doc of 10 November 2026.

B · In-scheduler variant
The controller runs inside the real SGLang scheduler, with the other review recommendations applied.

The specimen

The values are the design doc's own worked example (page 3), so each frame maps back to it. They stay fixed through the whole walkthrough.

Cell and attempt	`C1`, attempt `a1`, SGLang v0.5.23, TP 1, no DP, page size 16
Point `P`, cached prefill	2 requests, 256 new tokens (128 each), 1,024 past KV tokens (512 each)
Point `D`, steady decode	2 requests, 0 new tokens, 1,024 past KV tokens (512 each)
Requests for `P`	Seeds `S1`, `S2` (512 tokens). Measured `M1`, `M2` (640 = 512 prefix + 128 suffix).
Requests for `D`	`D1`, `D2` with 511-token prompts
State fields	`cache`: tokens held in the prefix cache per seed. `live`: requests holding KV, with their sequence length. `kept`: retained samples. `done`: points completed.

Route map

#	STAGE	A · ORIGINAL	B · IN-SCHEDULER	STATUS
1	Plan and freeze	AISim plan, attempts, resume	Same, with the vLLM-only gates listed	Same
2	Launch	New launcher for a standalone process	The serving worker in benchmark mode	Changed
3	Initialize	Adapter replays the `load_model` order	Stock scheduler construction	Changed
4	Limits and grid	Pool and graph limits	Adds scheduler limits, reuses Dynamo's point schema	Same rule
5	Global warmup	Warm shots through the adapter	One real request through the scheduler	Same rule
6	`P` seed	Direct forward plus native cache builder	Seed requests through the stock path	Same state
7	`P` measure	Operation wall time or DeviceTimer	Scheduler wall time plus GPU span	Changed
8	`D` seed and admission	Direct extend, first decode	Stock prefill, first decode step	Same arithmetic
9	`D` steady step	Second direct decode	Next scheduler loop iteration	Changed
10	Write evidence	New records and recorder	Dynamo rank-artifact schema v2	Changed
11	Finalize	Exit, disposable worker	Flush, write, stop. Serving handoff possible.	Same in practice
12	Validate and publish	Backend extension, new timing policy	Existing validator with a backend switch	Changed

Walkthrough

In each frame, the grey band holds what both designs do. The two lanes below it hold only what differs.

FRAME 1 OF 12

AISim plans and freezes the cell

Same

Before `plan=none`

After `plan_sha256=H attempt=a1 points=[P, D]`

BOTH DESIGNS

Planning, attempts, smoke, resume and retries are AISim's existing machinery. The plan hash and attempt identity stay with AISim.

	A · Original	B · In-scheduler
New work	"SGLang admission, launch arguments, probe and producer contract" (design doc, page 4).	The same work, named gate by gate: backend choices (`cli.py:27`), planner gate (`planner.py:731`), launch builder (`fpm_builder.py:72`), runtime probe (`runtime_probe.py:115`).

FRAME 2 OF 12

The worker is launched

Changed

BOTH DESIGNS

One worker per cell. AISim stops it after collection.

	A · Original	B · In-scheduler
Command	A new benchmark process with its own launch arguments and producer contract.	The serving worker, `python -m dynamo.sglang` with `--benchmark-mode agg`, using the flag names the vLLM benchmark already has (`--benchmark-points-file`, `--benchmark-warmup-iterations`, `--benchmark-timeout`, `--benchmark-output-path`).
How the controller gets in	It is the process.	A `Scheduler` subclass installed through `Engine.run_scheduler_process_func`. Dynamo's SGLang worker already overrides this hook.

Evidence: Engine.run_scheduler_process_func; `dynamo/sglang/nixl_telemetry.py:237`.

FRAME 3 OF 12

The runtime initializes Changed

After `cache={} live=[] kept=[] done=0/2`

BOTH DESIGNS

The same model, memory pools, attention backends and CUDA graphs get built.

	A · Original	B · In-scheduler
Init order	The adapter replays `one_batch.load_model`: parallel runtime, layer runtime, `ModelRunner`, pools, attention backends, graphs. The adapter owns this order and has to track it across SGLang releases.	Stock `Scheduler.__init__` inside `run_scheduler_process`. SGLang owns the order.
Prefix cache	Built separately with the native cache builder, because `one_batch.extend` uses a dummy cache.	The serving radix cache.
Graph policy	The adapter must stop `one_batch.main` from raising the decode graph batch limit.	The customer's configuration, untouched.

Evidence: load_model, dummy cache in extend, one_batch.main, run_scheduler_process.

FRAME 4 OF 12

Limits are resolved and the grid agreed Same rule

After `grid=[P, D] grid_digest=G`

BOTH DESIGNS

The grid is built from observed capacity and its digest is agreed before any point runs. An invalid explicit point fails. An invalid generated point is skipped and recorded.

	A · Original	B · In-scheduler
Limits	Native pool and graph limits (design doc, page 4).	Adds the scheduler's own limits: `chunked_prefill_size`, `max_prefill_tokens`, `max_running_requests`, `prefill_max_requests`.
Point schema	A new `CaseSpec` record.	Dynamo's `benchmark_points.py` schema and digest helpers, moved to shared code.

Evidence: prefill batch limits.

FRAME 5 OF 12

Global warmup `Same rule`

BOTH DESIGNS

Warmup timings are discarded. The executed count is recorded, not only the requested one.

	A · Original	B · In-scheduler
Mechanism	The controller runs warm shots through the adapter.	One real request through the scheduler, as vLLM does: a 256-token prefill, then the configured number of decode steps.

FRAME 6 OF 12

P seeds the prefix, untimed `Same state`

```
Before   cache={} live=[]
After    cache={S1:512, S2:512} live=[]
```

BOTH DESIGNS

A real, untimed prefill puts 512 tokens per request in the cache.

	A · Original	B · In-scheduler
How	Build `S1`, `S2`, call `prepare_for_extend` and `ModelRunner.forward`, then checkpoint into the separately built cache.	Enqueue `S1`, `S2` with one output token. The stock result path checkpoints them into the radix cache.
Provenance label	"Real prefix", kept distinct from synthetic state.	`real_prefix`, one of the four values AISim already validates.

Evidence: release_kv_cache, build_kv_cache.

FRAME 7 OF 12

P warm shot, measured shot, release `Changed`

```
Before   cache={S1:512, S2:512} kept=[] done=0/2
After    cache={S1:512, S2:512} live=[] kept=[P] done=1/2
```



Drawn to scale. 2 × 640 = 1,280 tokens = 1,024 matched + 256 computed in the measured forward.

BOTH DESIGNS

A warm shot, then the measured shot. Each request must match exactly 512 cached tokens. The 128-token suffix is never inserted into the cache. Both designs flush afterwards, so `cache={}` before `D`.

	A · Original	B · In-scheduler
Execute	Fresh `M1`, `M2`, match 512 each, `prepare_for_extend`, forward.	Enqueue `M1`, `M2` while the loop is held, so both land in one prefill batch. Stock `run_batch` and `process_batch_result` do the rest.
Timed interval	Synchronized wall time around the call, or the DeviceTimer span.	Both recorded: scheduler wall time, from `batch.launch_ts` to the start of result processing, and the native GPU span.
Actual shape	Checked by the new prefix provider.	Read from native forward-pass metrics: 2 prefill requests, 256 prefill tokens, 1,024 prefill KV tokens.
Keeping the suffix out	Release without insert.	Set `req.skip_radix_cache_insert` on the request, or give each shot a different suffix.

Evidence: run_batch, process_batch_result, shape metrics, FpmTiming, no-insert flag.

FRAME 8 OF 12

`D` seed and admission step, not retained

`Same arithmetic`

```
Before   cache={} live=[] kept=[P] done=1/2
After    live=[D1:512, D2:512] kept=[P] done=1/2
```

BOTH DESIGNS

The admission step reads 511 past tokens per request and ends at sequence length 512, which fills page 32. Its timing is dropped.

	A · Original	B · In-scheduler
How	Untimed `extend` on the 511-token prompts, then one `decode`.	Enqueue `D1`, `D2` with three output tokens and EOS ignored. Stock prefill runs, then the first decode step.

FRAME 9 OF 12

D steady step, release

Changed

```
Before    live=[D1:512, D2:512] kept=[P] done=1/2
After     live=[] kept=[P, D] done=2/2
```

page 32, tokens 497 to 512 page 33



from the 511-token prompt token 512, written by the admission step

token 513, written by the steady step

One request's KV slots at the page boundary. The steady step is the first forward to touch page 33.

BOTH DESIGNS

Each request reads 512 past tokens, moves to length 513 and opens page 33. This is the retained sample: 2 requests, 1,024 past KV tokens.

	A · Original	B · In-scheduler
Execute	A second direct `decode`.	The next scheduler loop iteration.
Timed interval	Synchronized wall time around the call.	The interval between consecutive results when overlap scheduling is on, the counterpart of vLLM's `inter_output`. Launch to result when it is off.
Coordinate	Computed by the runner: 512 + 512 = 1,024.	Native metrics report 513 + 513 = 1,026, because `prepare_for_decode` has already advanced the length. Subtract the batch size to get 1,024.

In lane B the admission step natively reports 512 + 512 = 1,024. Selecting the sample by the native value would keep the wrong step.

Evidence: prepare_for_decode advances the length at line 3787. The conversion comes from reading source and needs a GPU trace.

FRAME 10 OF 12

Reduce and write rank evidence

Changed

BOTH DESIGNS

Raw evidence is kept. One retained sample per point.

	A · Original	B · In-scheduler
Records	New `RawSample` and `CaseResult` records and a new recorder.	Dynamo's rank-artifact schema v2, unchanged: `results`, `iteration_groups`, `run_id`, `grid_digest`, `coverage`, `timing`.
Rank reduction	To be specified for each timing policy.	Group time is the maximum over ranks, as today.

FRAME 11 OF 12

The worker finalizes

Same in practice

BOTH DESIGNS

AISim treats the worker as disposable and stops it.

	A · Original	B · In-scheduler
End state	Write artifacts, tear down, exit.	Flush the cache, write the artifact, then stop.
Serving handoff	A fresh worker only.	Possible in the same worker, still optional.

FRAME 12 OF 12

AISim validates and publishes

Changed

After

```
row P: batch_size=2 total_prefill_tokens=256 total_kv_read_tokens=1024
row D: batch_size=2 total_prefill_tokens=0 total_kv_read_tokens=1024
```

BOTH DESIGNS

Complete coverage and matching identity are required before publication.

	A · Original	B · In-scheduler
Validation	"AISim plus backend extension".	The existing `native_artifact.py` checks, with the `runtime.backend == "vllm"` test at line 383 switched on backend.
Per-point check	New.	`_expected_scheduled` at line 416 already requires exactly the `P` and `D` shapes above.
Timing policy	A new standalone policy in the writer and the Rust reader.	The existing row shape, plus an explicit `timing_boundary` column.

Unhappy paths

Each branch starts from a frame above. In the cache-miss and deadline branches both designs end the same way: the cell is incomplete, AISim keeps the evidence and does not publish it as complete.

BRANCH FROM FRAME 4

An explicit point asks for 16,384 new tokens while the prefill chunk is 8,192

	A · Original	B · In-scheduler
Outcome	The point passes pool and graph limits and runs as one forward, a shape chunked serving would not produce.	Rejected at grid build. If it slipped through, the scheduler would chunk it and the shape check would fail.

BRANCH FROM FRAME 7

`M2` matches 496 cached tokens, not 512

	A · Original	B · In-scheduler
Detection	The prefix provider marks the point invalid.	Metrics show 1,008 prefill KV tokens instead of 1,024. The controller may re-stage once, as vLLM does.

BRANCH BEFORE FRAME 8

The soft deadline fires

BOTH DESIGNS

Stop at the point boundary. The attempt is partial, with `P` done and `D` missing. Resume stays bound to the frozen plan.

BRANCH FROM FRAME 7, WITH TP ABOVE 1

One rank's forward raises

	A · Original	B · In-scheduler
Outcome	Needs new failure propagation. `one_batch.main` only joins its rank processes.	The stock path signals the parent and the worker dies. AISim marks the attempt failed.

Evidence: rank processes in one_batch.main, scheduler failure path.

Invariant ledger

RULE	ESTABLISHED	CHECKED	A AGAINST B
Actual shape equals planned shape	Frame 4	Frames 7, 9, 12	Same rule. B reads native metrics.
Setup, admission and warmup never enter a retained sample	Frames 5, 6, 8	Frames 7, 9	Same
State provenance is truthful	Frames 6, 8	Frame 12	Same. B uses AISim's existing labels.
Grid and point order are agreed	Frame 4	Frame 12	Same
Everything is released before the next point	Frames 7, 9	Frame 11	Same rule. B uses the stock release path.
Complete coverage before publication	Frame 1	Frame 12	Same

Code map

Lane A reuses

- one_batch.py helpers
- prepare_for_extend and prepare_for_decode
- DeviceTimer

Lane B reuses

- The scheduler event loops and recv_requests
- get_new_batch_prefill and flush_cache
- Native forward-pass metrics

AISim, both lanes

- `collector/fpm_forward/native_artifact.py`
- `collector/fpm_forward/database.py` and `planner.py`
- `perf_database/fpm_forward.rs`, the Rust reader

Boundaries and unknowns

- The specimen is TP 1 without DP. With DP attention both designs need a per-point barrier across ranks, like vLLM's synchronizer. That is outside this journey.
- Request injection in lane B is undecided. One option is the engine's front door from the Dynamo worker. The other is direct enqueue at `recv_requests` on rank 0, which needs handling for outputs of requests the tokenizer manager does not know.
- Two things need a GPU trace: the scheduler wall-time interval under overlap scheduling, and the 1,026 to 1,024 decode conversion.

Sources read: SGLang at the v0.5.23 tag, Dynamo at 95b9d56 and AISimulate at 925c66ec. These are not the revisions the design doc pins, so Dynamo and AISim line numbers may differ slightly. Specimen values are illustrative.